```
  useEffect(() => { #A
    savedCallback.current = callback;
  }, [callback]);
  useEffect(() => { #B
    function tick() {
      if (savedCallback.current) {
        savedCallback.current();
      }
    }
    if (delay !== null) {
      let id = setInterval(tick, delay);
      return () => clearInterval(id);
    }
  }, [delay]);
}
```

Listing 3.13 demonstrates the `Timer` component, located in `use-cases/Timer.tsx`, that uses the `useInterval` hook to implement a timer that increments every second when running.

Listing 3.13 Implementation of the Timer component

```
const Timer: React.FC = () => {
  const [seconds, setSeconds] = useState<number>(0); #A
  const [isRunning, setIsRunning] = useState<boolean>(false); #B
  useInterval( #C
    () => {
      setSeconds((prev: number) => prev + 1);
    },
    isRunning ? 1000 : null #D
  );
  const startTimer = () => setIsRunning(true); #E
  const pauseTimer = () => setIsRunning(false);
  const resetTimer = () => {
    setIsRunning(false);
    setSeconds(0);
  };
  return (
    <div>
      <h1>Timer: {seconds} seconds</h1>
      <button onClick={startTimer} disabled={isRunning}>
        Start
      </button>
      <button onClick={pauseTimer} disabled={!isRunning}>
        Pause
      </button>
```